

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

NAME:K.DEEPIKA

REGNO:192371042

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS:

Total Marks: 25

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I _____ (K.Deepika / 192371042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.DEEPIKA

ANSWERS:

1.Question 1 — REST vs SOAP for a Mobile and Web Application (5 Marks)

1.1 Analysis

REST (Representational State Transfer) treats every capability as a resource, addressed by a URI and manipulated with standard HTTP verbs (GET, POST, PUT, DELETE). It's stateless, lightweight, and typically exchanges JSON. SOAP (Simple Object Access Protocol) is a strict XML-based messaging protocol with its own envelope/header/body structure, usually described formally by a WSDL contract, and carries built-in extensions like WS-Security, WS-AtomicTransaction, and WS-ReliableMessaging.

Criterion	REST	SOAP
Data format	JSON (lightweight), also XML	XML only (verbose envelope)
Transport	HTTP/HTTPS	HTTP, SMTP, TCP, JMS – transport independent
Performance	Faster – smaller payload, less parsing	Slower – large XML payload, strict parsing
Scalability	Stateless → scales horizontally easily, cache-friendly	Harder to scale – heavier processing per call
Security	Relies on HTTPS/TLS, OAuth2, JWT	Built-in WS-Security (message-level encryption/signatures)
Mobile suitability	Excellent – low bandwidth, simple parsing	Poor – heavy for constrained devices
Contract	No formal contract (OpenAPI optional)	Formal WSDL contract, strongly typed

1.2 Selected Approach & Design

REST is selected as the primary architecture, since both mobile and web clients need fast, low-bandwidth, cacheable JSON responses over HTTPS.



1.3 Justification

- Performance: JSON payloads are 2–3x smaller than equivalent SOAP/XML envelopes, cutting latency on mobile networks.
- Scalability: REST is stateless, so any server instance can serve any request — enabling simple horizontal scaling behind a load balancer plus HTTP caching (ETags, CDN).
- Security: HTTPS/TLS combined with OAuth 2.0/JWT tokens gives adequate transport and application-level security for consumer apps.
- Development speed: REST is simpler to design/test/consume from mobile SDKs and JavaScript.

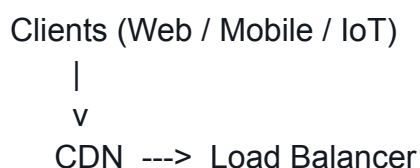
SOAP still wins where a rigid, machine-verifiable contract and message-level security is mandatory — e.g., enterprise banking transactions (see Q3).

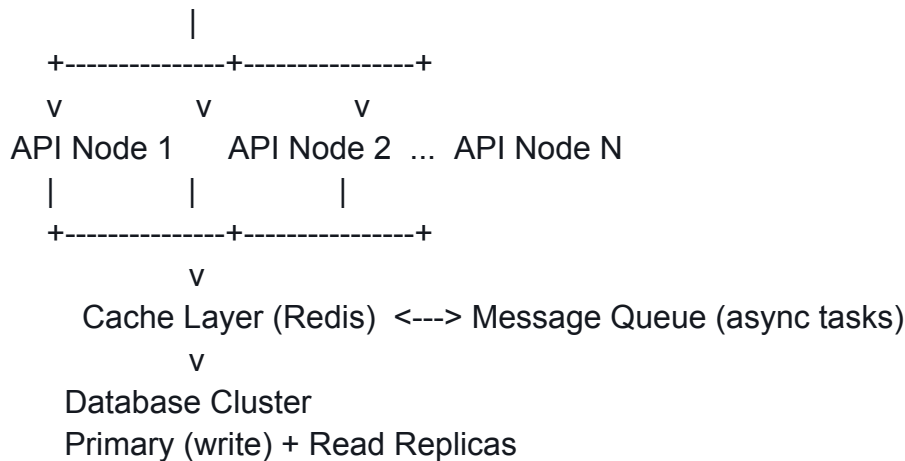
Question 2 — Scalable Service Architecture for High Concurrency (5 Marks)

2.1 Baseline

A single client → single server → single database model is a bottleneck at scale: the server and database are both single points of failure/contention.

2.2 Proposed Scalable Design





2.3 Alternative Design to Improve Scalability

- Horizontal scaling: multiple stateless API instances behind a load balancer instead of one large server.
- Caching: Redis/Memcached for frequent reads, CDN for static assets — cuts direct DB load significantly.
- Database scaling: read replicas for read-heavy traffic; shard by a partition key (e.g., user-id range) if writes are the bottleneck.
- Microservices decomposition: split the monolith into independent services (users, orders, payments) scaled/deployed separately.
- Asynchronous processing: offload slow operations (emails, reports) to a message queue (Kafka/RabbitMQ) so the API responds immediately.

2.4 Justification

This removes single points of failure and lets each layer scale independently. Caching and read replicas absorb most read traffic, while the queue smooths spikes in write-heavy/long-running operations, keeping latency low as concurrency grows.

Question 3 — Secure Communication Protocol for a Banking Application (5 Marks)

3.1 Analysis of Protocols

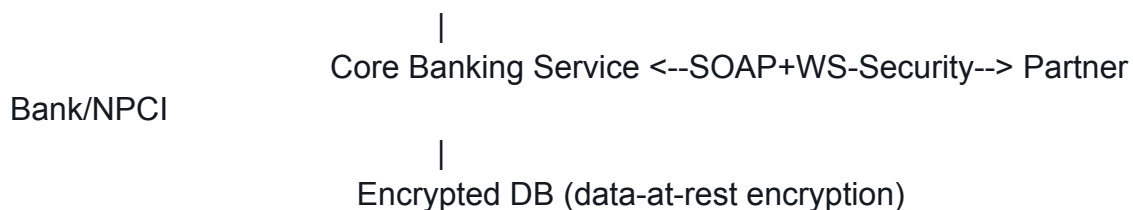
Protocol	Security	Reliability	Performance
Plain HTTP	None – clear text	Good	Fast but unsafe
HTTPS (TLS 1.2/1.3)	Strong transport encryption + cert auth	Good	Slight handshake overhead

SOAP + WS-Security over HTTPS	Message-level encryption + signatures + transport encryption	Very high (WS-ReliableMessaging)	Slower – larger XML payload
REST + OAuth 2.0/JWT over HTTPS	Token-based auth + transport encryption	High	Fast, lightweight
SFTP	Strong encryption, file transfer only	High for batch transfer	Not suited to real-time API calls

3.2 Selected Protocol

HTTPS (TLS 1.3) is mandatory for every communication. Combine SOAP + WS-Security for core, high-value transactions (fund transfers, statements) between bank and partner systems, and REST + OAuth 2.0/JWT for customer-facing mobile/web channels.

Customer App --HTTPS(TLS1.3)+OAuth2/JWT--> API Gateway (rate limiting, token validation)



3.3 Justification

- Security: TLS 1.3 protects data in transit; WS-Security adds message-level signing/encryption that survives intermediaries.
- Reliability: WS-ReliableMessaging guarantees exactly-once delivery for critical transactions like fund transfers.
- Performance: REST/JSON + OAuth2 keeps low-risk, high-volume operations (balance enquiry) fast, while heavier SOAP is reserved for sensitive inter-bank operations.
- Compliance: satisfies typical PCI-DSS/RBI requirements for transport + message-level protection.

Question 4 — Third-Party API Integration Strategy (Payment Gateway / Maps)
(5 Marks)

4.1 Integration Design

Never call third-party APIs directly from client apps — route all outbound calls through the application's own backend.

Client App --(1. Request)--> Application Backend (Integration Layer)
--(2. Signed/authenticated request)-->
Third-Party Service (Payment Gateway/Maps API)
--(3. Response/Webhook)-->
Application Backend --(4. Verify + persist)--
--(5. Final status)--> Client App

4.2 Request Handling

- All calls made server-side, never from browser/mobile — credentials never exposed.
- Built via vendor SDK or a thin REST wrapper, with strict timeouts and retry limits.

4.3 Authentication

- API key/secret stored in a server-side secrets manager (never in client code or source control).
- OAuth 2.0 client-credentials flow preferred over static keys where supported.
- Webhook callbacks verified via the provider's HMAC signature before being trusted.

4.4 Response Processing

- Responses validated against an expected schema; unexpected fields/error codes logged.
- Idempotency keys on payment requests so a network retry can't duplicate a charge.
- Successful results persisted locally before confirming success to the client.

4.5 Alternative Approach

- Asynchronous, event-driven integration: backend queues the request, an async worker calls the third-party API, and the result is pushed to the client via webhook + WebSocket/push notification, instead of a blocking synchronous call.
- Circuit breaker + local cache (e.g., cache geocoding results) so a slow/failing third-party API degrades gracefully instead of blocking the request path.

This improves reliability (an outage doesn't block the core app) and efficiency (repeated/predictable requests served from cache).

Question 5 — Error Handling and Fault Tolerance for a Web Service (5 Marks)

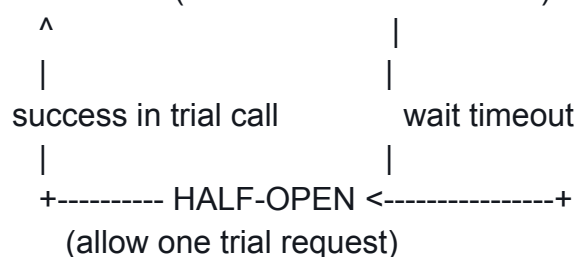
5.1 Error Handling Design

- Input validation at the API boundary — reject malformed requests early (HTTP 400) with a structured error body (code, message, field).
- Consistent error envelope across all endpoints: `{ "error": { "code": "...", "message": "..." } }`.
- Correct HTTP status codes: 400 bad input, 401/403 auth failures, 404 missing resource, 429 rate limit, 500/503 server/downstream failure.
- Centralized exception handling (global exception handler/middleware).
- Structured logging + monitoring/alerting (ELK, Prometheus + Grafana).

5.2 Fault Tolerance Mechanism

Circuit-breaker pattern:

CLOSED --(failures exceed threshold)--> OPEN



- Retry with exponential backoff + jitter for transient errors, capped at a few attempts.
- Timeouts on every outbound call so one slow dependency can't stall the thread pool.
- Fallback/graceful degradation — serve cached or default data when a downstream service is unavailable.
- Bulkhead isolation — separate connection/thread pools per dependency, so one failure can't starve others.

5.3 Alternative Strategy

- Asynchronous, queue-based processing: requests placed on a message queue (Kafka/RabbitMQ) with dead-letter queues for repeatedly-failing messages, retried safely without blocking the caller.
- Health checks + auto-restart: liveness/readiness probes (Kubernetes-style) automatically remove/restart unhealthy instances.

5.4 Justification

Circuit breakers, timeouts, retries with backoff, bulkheads, and an async fallback path let a single component's failure degrade gracefully instead of cascading into an outage — users get a cached response or a clear error message while the issue is fixed.

Summary Table

Q	Scenario	Chosen Design	Key Alternative
1	Mobile + Web service	REST over HTTPS with JSON	SOAP for contract-heavy, high-security cases
2	High-concurrency platform	Load-balanced API tier + cache + DB replicas	Microservices + async queue for further scale
3	Banking communication	HTTPS/TLS + SOAP/WS-Security (core) + REST/OAuth2 (customer channel)	Full REST + OAuth2/JWT for lighter operations
4	Third-party integration	Server-side gateway with OAuth2/API-key + idempotency	Async, event-driven integration with caching
5	Error handling / fault tolerance	Centralized handling + circuit breaker + retries + timeouts	Queue-based async processing + health-check auto-restart

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect